



PRACTICA 2 – POWERSHELL

Introducción:

1. Tipos de Datos

1. Array:

En PowerShell, un array es una estructura que permite almacenar una colección de elementos, los cuales pueden ser de cualquier tipo de datos, incluyendo números, cadenas de texto, objetos e incluso otros arrays. Los arrays en PowerShell son de tamaño fijo una vez creados, pero es posible crear nuevos arrays basados en los existentes para agregar o eliminar elementos

- **Crear un array vacío:** Para inicializar un array vacío, puedes utilizar el siguiente comando:

```
$miArray = @()
```

- **Crear un array con elementos predefinidos:** Este comando crea un array que contiene los números del 1 al 5

```
$miArray = @(1, 2, 3, 4, 5)
```

- **Agregar elementos a un array existente:** Para añadir elementos a un array ya existente, puedes utilizar el operador +=:

```
$miArray += 6
```

```
$miArray += 7
```

- **Acceder a elementos de un array:**

```
$primerElemento = $miArray[0]
```

```
$segundoElemento = $miArray[1]
```

- **Recorrer un array:** Para iterar sobre todos los elementos de un array, puedes usar un bucle foreach:

```
foreach ($elemento in $miArray) {  
    Write-Output $elemento  
}
```

- **Obtener la longitud de un array:** Para conocer el número de elementos en un array, utiliza la propiedad Length:

2. Estructuras de control, condicionales y bucles

1. Estructura condicional if

La instrucción if se utiliza para evaluar una o más condiciones y ejecutar bloques de código en función de si estas condiciones son verdaderas o falsas.

**Sintaxis:**

```

if (<condición1>) {
  # Bloque de código si la condición1 es verdadera
} elseif (<condición2>) {
  # Bloque de código si la condición1 es falsa y la condición2 es verdadera
} else {
  # Bloque de código si todas las condiciones anteriores son falsas
}

```

Ejemplo:

```

$numero = 10

if ($numero -gt 0) {
  Write-Output "El número es positivo."
} elseif ($numero -lt 0) {
  Write-Output "El número es negativo."
} else {
  Write-Output "El número es cero."
}

```

En este ejemplo, se evalúa si la variable \$numero es mayor que cero, menor que cero o igual a cero, y se muestra un mensaje correspondiente para cada caso.

2. Bucle foreach

El bucle foreach se emplea para iterar sobre cada elemento de una colección, como una matriz o una lista de objetos.

Sintaxis:

```

foreach ($elemento in $colección) {
  # Bloque de código a ejecutar para cada elemento
}

```

Ejemplo:

```

$frutas = @("Manzana", "Banana", "Cereza")

foreach ($fruta in $frutas) {
  Write-Output "Fruta: $fruta"
}

```

Este script recorre la matriz \$frutas y muestra el nombre de cada fruta en la consola.

Uso de ForEach-Object

Además del bucle foreach, PowerShell ofrece el cmdlet ForEach-Object, que se utiliza principalmente en la canalización para procesar cada elemento que pasa a través de ella.

Ejemplo:

```

1..5 | ForEach-Object {
  Write-Output "Número: $_"
}

```



En este caso, la secuencia de números del 1 al 5 se canaliza a ForEach-Object, que ejecuta el bloque de script para cada número, representado por la variable automática \$_.

3. Item.

El cmdlet **New-Item** se utiliza para crear nuevos elementos en diferentes ubicaciones, y los tipos de elementos que se pueden crear dependen de la ubicación especificada. A continuación, se detallan algunos de los tipos más comunes:

- **Sistema de archivos:**
 - **Archivos:** Para crear un nuevo archivo, se utiliza el parámetro **-ItemType** con el valor "file".
 - **Directorios (carpetas):** Para crear un nuevo directorio, se especifica **-ItemType** con el valor "directory".
- **Registro de Windows:**
 - **Claves del registro:** Al trabajar con el registro, **New-Item** puede crear nuevas claves.
 - **Entradas del registro:** También es posible crear nuevas entradas dentro de las claves existentes.
- **Enlaces simbólicos:**
 - **Enlaces simbólicos a archivos o carpetas:** Utilizando **-ItemType** con el valor "SymbolicLink" y el parámetro -Target, se pueden crear enlaces simbólicos que apuntan a archivos o directorios específicos.

A continuación, se detallan los parámetros más comunes que puedes utilizar con New-Item para especificar la ruta y otras propiedades del elemento a crear:

- **-Path:** Especifica la ruta donde se creará el nuevo elemento. Puede incluir el nombre del nuevo elemento directamente en esta ruta.

New-Item -Path "C:\Ruta\NuevaCarpeta"

Este comando crea una nueva carpeta llamada "NuevaCarpeta" en "C:\Ruta".

- **-Name:** Define el nombre del nuevo elemento. Se utiliza junto con -Path cuando la ruta no incluye el nombre del elemento.

New-Item -Path "C:\Ruta" -Name "Archivo.txt" -ItemType "File"

Este comando crea un archivo llamado "Archivo.txt" en "C:\Ruta".

- **-ItemType:** Especifica el tipo de elemento a crear. Los valores comunes incluyen:
 - "File": Crea un archivo.
 - "Directory": Crea una carpeta.
 - "SymbolicLink": Crea un enlace simbólico.

New-Item -Path "C:\Ruta" -Name "NuevaCarpeta" -ItemType "Directory"

Este comando crea una nueva carpeta llamada "NuevaCarpeta" en "C:\Ruta".



- **-Value:** Establece el contenido inicial del nuevo elemento. Es útil al crear archivos o entradas en el registro.

New-Item -Path "C:\Ruta" -Name "Archivo.txt" -ItemType "File" -Value "Contenido inicial del archivo."

Este comando crea un archivo llamado "Archivo.txt" con el contenido "Contenido inicial del archivo."

- **-Force:** Permite la creación de elementos en rutas que no existen actualmente, creando las carpetas necesarias. También sobrescribe elementos existentes si es necesario.

New-Item -Path "C:\Ruta\NuevaCarpeta\Archivo.txt" -ItemType "File" -Force

Este comando crea un archivo llamado "Archivo.txt" en "C:\Ruta\NuevaCarpeta". Si "NuevaCarpeta" no existe, se creará automáticamente.

- **-Target:** Especifica el destino de un enlace simbólico cuando -ItemType se establece en "SymbolicLink".

New-Item -Path "C:\Ruta\Enlace" -ItemType "SymbolicLink" -Target "C:\Destino\ArchivoOriginal.txt"

4. Set-location

En PowerShell, el cmdlet **Set-Location** se utiliza para cambiar la ubicación de trabajo actual a una ruta especificada. Esta ruta puede ser un directorio en el sistema de archivos, una clave del registro u otra ubicación proporcionada por un proveedor de PowerShell.

Set-Location -Path "Ruta\Deseada"

Navegación en el historial de ubicaciones:

PowerShell mantiene un historial de las últimas 20 ubicaciones de trabajo. Puedes navegar por este historial utilizando los símbolos - y +. El símbolo - te lleva a la ubicación anterior, mientras que + te lleva a la siguiente ubicación en el historial.

Set-Location -
Set-Location +

O utilizando el alias cd:

cd -
cd +

Condicional if para verificar si una ruta existe:

```
$ruta = "C:\Ejemplo\Directorio"

if (Test-Path -Path $ruta) {
    Write-Output "La ruta existe."
} else {
    Write-Output "La ruta no existe."
}
```



5. Import-csv

Este cmdlet lee el contenido del archivo CSV y convierte cada fila en un objeto de PowerShell con propiedades correspondientes a los encabezados de columna del archivo.

```
$usuarios = Import-Csv -Path "C:\ruta\al\archivo\usuarios.csv"
```

En este ejemplo, el contenido del archivo usuarios.csv se almacena en la variable \$usuarios. Cada fila del CSV se convierte en un objeto con propiedades basadas en los nombres de las columnas.

Estructura del archivo CSV:

Para que Import-Csv funcione correctamente, el archivo CSV debe tener una estructura adecuada:

- **Encabezado de columnas:** La primera fila debe contener los nombres de las columnas, que se convertirán en las propiedades de los objetos en PowerShell.
- **Datos de usuarios:** Las filas subsiguientes deben contener los datos correspondientes a cada usuario, alineados con los encabezados de columna.

Ejemplo de archivo CSV:

```
Nombre,Apellido,CorreoElectronico,IDUsuario  
Juan,Pérez,juan.perez@ejemplo.com,jperez  
María,García,maria.garcia@ejemplo.com,mgarcia
```

En este ejemplo, el archivo CSV tiene cuatro columnas: Nombre, Apellido, CorreoElectronico e IDUsuario. Cada fila siguiente representa un usuario con sus respectivos datos.

Acceso a los datos en PowerShell:

Después de importar el CSV, puedes iterar sobre los objetos y acceder a sus propiedades:

```
foreach ($usuario in $usuarios) {  
    Write-Output "Nombre completo: $($usuario.Nombre) $($usuario.Apellido)"  
    Write-Output "Correo electrónico: $($usuario.CorreoElectronico)"  
    Write-Output "ID de usuario: $($usuario.IDUsuario)"  
    Write-Output ""  
}
```

Este script recorrerá cada objeto en \$usuarios y mostrará la información de cada usuario.

Consideraciones adicionales:

- **Delimitador:** Por defecto, Import-Csv asume que las columnas están separadas por comas. Si tu archivo utiliza un delimitador diferente, como punto y coma, puedes especificarlo con el parámetro -Delimiter:

```
$usuarios = Import-Csv -Path "C:\ruta\al\archivo\usuarios.csv" -Delimiter ";"
```

- **Codificación:** Si el archivo CSV utiliza una codificación específica, como UTF-8, puedes especificarlo con el parámetro -Encoding:

```
$usuarios = Import-Csv -Path "C:\ruta\al\archivo\usuarios.csv" -Encoding UTF8
```



Secuencia / Desarrollo

1. Ejercicio 1: Abre el Entorno de Scriping Integrado (ISE) de Windows Powershell . Crea en Windows Server una carpeta llamada Practica2-PS

1. Crea el script creaCarpeta.ps1 para crear una carpeta en una ruta específica

New-Item -Path "C:\Ruta\Deseada" -Name "NombreCarpeta" -ItemType "Directory"

2. Crea el script info.ps1 que creará el fichero INFO1_1.txt cuyo contenido será:

Fecha - Hora : dd/mm/aa - hh:mm:ss

Nombre del PC: xxxx

Usuario actual: yyy

tomando dd/mm/aa , hh:mm:ss, xxx e yyy los valores correspondientes.

3. Crea el script verdir.ps1 que listará el contenido de un directorio:
Para ello se le solicitará al usuario un nombre a la carpeta.. Si no existe se debe mostrar un mensaje de error. Si todo correcto se lista contenido del directorio.
4. Tomando como base el **ejercicio 1**, debes crear múltiples carpetas. Para ello debes especificar una lista de rutas en un array y utilizar la sentencia necesaria para que funcione correctamente
5. Tomando como base el **ejercicio 4** añade las sentencias necesarias para comprobar su existencia.
6. A continuación añadir al ejercicio anterior la creación de usuarios almacenados en un array y sus carpetas personales.
7. Repetir el ejercicio anterior pero en este caso vamos a leer los datos de los usuario de un fichero **user.csv** creado con anterioridad. Dicho fichero debe contener los campos citados a continuación.

Nombre, Apellido, CorreoElectronico e IDUsuario